P2918R1
Runtime format strings II

Published Proposal, 2023-07-15

Author:
Victor Zverovich

Audience:
LEWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

"Temporary solutions often become permanent problems." — Craig Bruce

§ 1. Introduction

[P2216] "std::format improvements" introduced compile-time format string checks which, quoting Barry Revzin, "is a fantastic feature" ([P2757]). However, due to resource constraints it didn't provide a good API for using formatting functions with format strings not known at compile time. As a workaround one could use type-erased API which has never been designed for that. This severely undermined safety and led to poor user experience. This paper proposes direct support

for runtime format strings which has been long available in the {fmt} library and its companion paper ([P2905]) fixes the safety issue.

2. Problems

[P2216] "std::format improvements" introduced compile-time format string checks for `std::format`. This obviously requires format strings be known at compile time. However, there are some use cases where format strings are only known at runtime, e.g. when translated through gettext ([GETTEXT]). One possible workaround is using type-erased formatting functions such as `std::vformat`:

```
std::string str = translate("The answer is {}.");
std::string msg = std::vformat(str, std::make_format_args(42));
```

This is not a great user experience because the type-erased API was designed to avoid template bloat and should only be used by formatting function writers and not by end users.

Such misuse of the API also introduces major safety issues illustrated in the following example:

```
std::string str = "{}";
std::filesystem::path path = "path/etic/experience";
auto args = std::make_format_args(path.string());
std::string msg = std::vformat(str, args);
```

This innocent-looking code exhibits undefined behavior because format arguments store a reference to a temporary which is destroyed before use. This has been discovered and fixed in [FMT] which now rejects such code at compile time.

3. Changes

- Added poll results.

4. Polls

LEWG poll results:

POLL: Send P2918R1 (Runtime Format Strings II) to Library for C++26.

```
SF  F  N  A SA
 4  6  0  0  0
```

Outcome: Unanimous consent in favor.

§ 5. Proposal

This paper proposes adding the `std::runtime_format` function to explicitly mark a format string as a runtime one and opt out of compile-time format string checks.

Before	After
`std::vformat(str, std::make_format_args(42));`	`std::format(std::runtime_format(str), 42);`

This improves usability and makes the intent more explicit. It can also enable detection of some lifetime errors for arguments ([P2418]). This API has been available in {fmt} since `consteval`-based format string checks were introduced ~2 years ago and usage experience was very positive. In a large codebase with > 100k calls of `fmt::format` only ~0.1% use `make_format_args`.

This was previously part of [P2905] but moved to a separate paper per LEWG feedback with the original paper focusing on the safety fix only.

§ 6. Impact on existing code

This paper adds a new API and has no impact on existing code.

§ 7. Wording

Change in [format.syn]:

```
namespace std {
  ...

  // [format.fmt.string], class template basic_format_string
  template<class charT, class... Args>
    struct basic_format_string;


  template<class charT> struct runtime-format-string {  // exposition-only
    basic_string_view<charT> str;  // exposition-only
  };

  runtime-format-string<char> runtime_format(string_view fmt) { return {fmt}; }
  runtime-format-string<wchar_t> runtime_format(wstring_view fmt) { return {fmt}; }

  ...
}
```

Change in [format.fmt.string]:

```
namespace std {
  template<class charT, class... Args>
  struct basic_format_string {
```

```
  private:
    basic_string_view<charT> str;         // exposition only

  public:
    template<class T> consteval basic_format_string(const T& s);
    basic_format_string(runtime-format-string<charT>&& s) : str(s.str) {}

    constexpr basic_string_view<charT> get() const noexcept { return str; }
  };
}
```

8. Implementation

The proposed API has been implemented in the {fmt} library ([FMT]).

9. Acknowledgements

Thanks to Mateusz Pusz for the suggestion to pass *runtime-format-string* by rvalue reference that improves safety of the API.

References

Informative References

[FMT]
Victor Zverovich; et al. *The fmt library*. URL: https://github.com/fmtlib/fmt

[GETTEXT]
Free Software Foundation. *gettext*. URL: https://www.gnu.org/software/gettext/

[P2216]
Victor Zverovich. *std::format improvements*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2216r3.html

[P2418]
Victor Zverovich. *Add support for `std::generator`-like types to `std::format`*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2418r2.html

[P2757]
Barry Revzin. *Type-checking format args*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2757r1.html

[P2905]
Victor Zverovich. *Runtime format strings*. URL: https://isocpp.org/files/papers/P2905R1.html